数据库恢复技术：事务、故障与检查点恢复

一、事务基础概念

1. 定义

事务是**用户定义的数据库操作序列**，是不可分割的工作单位，操作要么全执行、要么全不执行；

- 一个事务可对应一条/一组SQL语句或整个程序，**一个程序通常包含多个事务**；
- 事务是数据库**恢复**和**并发控制**的基本单位（核心考点）。

2. 事务的定义方式

（1）显式定义

核心关键字：

- `COMMIT`：事务正常结束，提交所有更新，将事务对数据库的修改**写回磁盘**；
- `ROLLBACK`：事务异常终止，**撤销所有已完成的更新操作**，回滚至事务开始状态。

（2）隐式定义

用户未显式定义时，由DBMS按默认规则**自动划定事务边界**（了解即可）。

3. 事务的ACID特性（核心）

事务的四大基本特性是保证数据一致性的基础，四者相互关联，原子性是一致性的前提：

特性	英文	核心含义
原子性	Atomicity	事务是逻辑工作单位，操作要么全做、要么全不做
一致性	Consistency	事务执行后，数据库从**一个一致性状态**到另一个一致性状态（无中间错误状态）
隔离性	Isolation	事务内部操作及数据对其他并发事务隔离，互不干扰
持久性	Durability	事务提交后，对数据库的修改**永久有效**，不受后续故障/操作影响

一致性示例：银行转账（A转1万至B）

- 正确操作：$A = A - 10000$ 且 $B = B + 10000$，数据库保持一致性；
- 异常情况：仅执行$A = A - 10000$，数据丢失，数据库处于不一致状态。

二、数据库故障类型（三类）

故障不可避免（硬件、软件、人为失误、恶意破坏），故障对数据库的影响分两种：**数据库本身被破坏/数据库未破坏但数据不正确**，三类故障的核心特征及成因如下：

1. 事务内部故障

- 分类：① 程序可处理的故障（如转账时余额不足，程序主动`ROLLBACK`）；② **非预期故障**（核心研究对象，如运算溢出、并发死锁被撤销、违反完整性约束）；
- 特点：仅影响单个事务，不破坏数据库，仅导致事务无法正常完成。

2. 系统故障（软故障）

- 定义：导致系统停止运转、需重新启动的事件，**不破坏数据库**，但影响所有正在运行的事务；
- 核心后果：**内存数据缓冲区信息全部消失**，未完成事务的更新可能已写入磁盘，已提交事务的更新可能仍在缓冲区未写入磁盘，最终导致数据不一致；
- 成因：CPU故障、操作系统/DBMS代码错误、系统断电、计算机病毒等。

3. 介质故障（硬故障/外存故障）

- 定义：外存设备损坏导致的故障，**直接破坏数据库或部分数据**；
- 特点：发生概率最低，但**破坏性最大**，影响所有存取故障数据的事务；
- 成因：磁盘损坏、磁头碰撞、强磁干扰、破坏硬盘数据的病毒等；
- 解决思路：依赖数据库备份（多副本存储，如大厂的异地容灾）。

三、数据库恢复策略（对应三类故障）

恢复的核心依托：**日志文件**（记录事务的所有更新操作）、**数据库后备副本**（介质故障专用），不同故障的恢复主体、操作、步骤不同，核心区分`UNDO`（撤销未完成事务的更新）和`REDO`（重做已提交事务的更新）。

1. 事务内部故障的恢复

- 恢复主体：DBMS**自动完成**，对用户完全屏蔽，无需人工干预；
- 核心操作：**仅执行UNDO**，撤销故障事务的所有更新操作；
- 恢复步骤：
  1. 从后向前**反向扫描日志文件**，查找该故障事务的所有更新记录；
  2. 对每个更新操作执行**逆操作**：
     - 插入操作→删除该记录；删除操作→插入该记录；修改操作→用**修改前值**替换修改后值；
  3. 持续扫描至找到**事务开始标记**，恢复完成。

2. 系统故障的恢复

- 恢复主体：DBMS在**系统重启时自动完成**，无需用户干预；
- 核心问题：① 未完成事务的更新可能已写入磁盘（需UNDO）；② 已提交事务的更新仍在缓冲区（需REDO）；
- 核心操作：**UNDO + REDO**；
- 恢复步骤：
  1. **正向扫描日志文件**，划分两个队列：
     - `REDO队列`：故障前已正常提交的事务（有`BEGIN TRANSACTION`和`COMMIT`记录）；
     - `UNDO队列`：故障时未完成的事务（仅有`BEGIN TRANSACTION`，无对应`COMMIT`记录）；

2. 对`UNDO队列`执行UNDO操作：反向扫描日志，对每个更新执行逆操作；
3. 对`REDO队列`执行REDO操作：正向扫描日志，**重新执行**所有登记的更新操作（将日志中**修改后值**写入数据库）。

3. 介质故障的恢复

- 恢复主体：**需DBA介入**（重装副本和日志文件），具体恢复操作由DBMS完成；
- 核心方法：**重装数据库后备副本 + 重做已完成事务**；
- 恢复步骤：
  1. **装入最新的数据库后备副本**，恢复至最近一次转储的一致性状态：
     - 静态副本：直接装入即为一致性状态；
     - 动态副本：需同时装入**转储时刻的日志文件副本**，用系统故障的恢复方法（UNDO+REDO）恢复至一致性；
  2. **装入转储结束时刻的日志文件副本**，重做已完成事务：
     - 正向扫描日志，找出故障前已提交的事务，记入`REDO队列`；
     - 正向扫描日志，对`REDO队列`执行REDO操作，将更新后值写入数据库。

四、具有检查点的恢复技术（优化版）

1. 引入原因

基础恢复策略需**扫描整个日志文件**，重做/撤销大量操作，耗时耗资源；检查点技术通过**划定恢复起点**，减少日志扫描范围和重做/撤销操作，**大幅提升恢复效率。**

2. 核心新增文件/记录

（1）检查点记录

写入日志文件，内容包括：

- 建立检查点时刻**所有正在执行的事务清单**；
- 这些事务**最近一次日志记录的地址。**

（2）重新开始文件

独立文件，核心内容：**记录每个检查点记录在日志文件中的地址**（恢复时先查此文件，快速定位检查点）。

3. 检查点的动态建立（DBMS周期性执行）

建立检查点的过程即**保存数据库一致性状态**的过程，步骤固定：

1. 将**当前日志缓冲区的所有记录**写入磁盘的日志文件；
2. 在日志文件中**写入一个检查点记录**；
3. 将**当前数据缓冲区的所有数据**写入磁盘的数据库；
4. 将**检查点记录在日志文件中的地址**写入重新开始文件。

建立频率：① 定期（如每5分钟、每1小时）；② 不定期（如日志文件写满50%时）。

4. 基于检查点的恢复策略（核心：划分事务状态）

系统故障时，恢复子系统先从**重新开始文件**找到**最近的检查点记录地址**，再在日志文件中定位该检查点，将检查点前后的事务分为五类（以检查点CP、故障点F为界），仅对部分事务执行UNDO/REDO：

事务状态	恢复操作	原因
检查点**前已提交**（T1）	无	检查点建立时，其更新已写入磁盘，无需重做
检查点前开始、故障前提交（T2）	REDO	提交后更新可能仍在缓冲区，未写入磁盘
检查点前开始、故障前未完成（T3）	UNDO	事务未完成，更新可能已写入磁盘，需撤销
检查点后开始、故障前提交（T4）	REDO	提交后更新可能仍在缓冲区，未写入磁盘
检查点后开始、故障前未完成（T5）	UNDO	事务未完成，更新可能已写入磁盘，需撤销

核心结论：仅对**检查点后提交**的事务执行REDO，对**故障前未完成**的事务执行UNDO，无需扫描检查点前的全量日志。